



BIRDCLEF+ 2026: AN AUTONOMOUS RESEARCH AGENT FOR BIRD SPECIES RECOGNITION

*Documenting the Discovery Process of an LLM-Driven
Agent under a Realistic CPU Compute Budget*

Advanced Predictive Analytics 2025/2026

Course Project — Track B

Spring 2026

Universidade Católica Portuguesa

Research performed by: Daniele Malerba / Irene Perdomo Bolaños / Miguel Afonso Magalhães

Project Summary

Competition:	BirdCLEF+ 2026 (Kaggle)
Reasoning core:	locally-hosted LLM (Google Gemma 4 E4B)
Best Kaggle public score (macro-AUC):	0.725
Best local soundscape macro-AUC:	0.8619 (single GPU model) / 0.6520 (CPU uniform ensemble)
Total agent experiments logged:	108
GitHub:	github.com/Daniele0302/birdclef-2026-agent

This project is an **autonomous research agent**: a local LLM that designs and runs deep-learning experiments on its own, on CPU and within a realistic time budget. The report focuses on what the agent discovered, not on a leaderboard score.

Over 108 experiments on BirdCLEF+ 2026, the agent tried training a CNN from scratch twice, abandoned it, settled on a frozen EfficientNetB0, and tuned the mel-spectrogram parameters until it reached **0.8533** focal validation macro-AUC. We log every iteration, track the failures it recovers from, and show how it stays within its compute budget (median experiment around 4.5 minutes, no transformer fine-tuning attempts).

We also retrained the same architecture on a GPU as a check that it scales: 0.725 on the Kaggle public leaderboard. We report this as confirmation that the agent’s choice was sound. The point of the project, though, is the agent itself.

1. Overview and Project Strategy

BirdCLEF+ 2026 asks for the probability of presence of each of 234 wildlife species in a 5-second audio window from passive-acoustic recordings in the Pantanal wetlands. The official metric is macro-averaged ROC-AUC, skipping classes with no positives.

We did not set out to hand-engineer the strongest possible model. The goal was to build an **autonomous research agent**, driven by a local LLM, that can explore this problem on a laptop CPU and arrive at a sensible solution by itself within a realistic time budget.

The approach has two parts. The agent runs many short experiments on small data subsets to find what kind of architecture, preprocessing and augmentation work; this is the practical reading of neural scaling laws (Kaplan et al. 2020; Hoffmann et al. 2022). Once it converges, we retrain its chosen architecture separately at larger scale, as a sanity check on the agent’s judgement.

The rest of the report covers the agent (Section 2), its 108 logged experiments (Section 3), the at-scale verification (Section 4), comparison with manual baselines (Section 5), course content used (Section 6), limitations (Section 7), and individual contributions (Section 8).

2. Agent Architecture

2.1 Modules

The agent is composed of six modules with strictly separated responsibilities:

- **agent.py** — the main loop that orchestrates one iteration: ask the LLM for parameters, run the experiment, log the result, ask the LLM for an analysis, repeat. Before the loop starts it triggers a one-shot *data exploration* pass (see Section 3.1).
- **data_exploration.py** — the agent’s *step 0*: loads the competition data, surfaces taxonomy/class counts, per-species file histograms, the species with zero focal audio, soundscape-label coverage, and an audio-format probe. Outputs a compact textual block that is then prepended to every LLM prompt, so the agent reasons over real dataset facts rather than generic priors.

- `llm_provider.py` — thin wrapper around the `ollama` Python client that talks to a local Ollama server on `localhost`; no network traffic leaves the machine. The wrapper exposes a single `call_llm(prompt)` function, so the underlying model can be swapped by changing one identifier. Final agent runs use **Google Gemma 4 E4B** (`gemma4:e4b` in Ollama, ≈ 10 GB, 4B active parameters), chosen to fit the laptop hardware available to the team.
- `code_executor.py` — a small helper that runs a piece of Python code in its own subprocess. The code is written to a temporary file and launched with `subprocess.run` using `sys.executable`, so the child uses the same Python interpreter (and therefore the same installed packages) as the agent. `stdout` and `stderr` are captured, and a configurable timeout aborts the run if it overshoots the per-experiment compute budget.
- `memory.py` — structured JSON log (`experiments/experiment_log.json`) with one entry per experiment: timestamp, full parameter set, success flag, metrics, the LLM’s analysis, and a truncated `stderr` snippet for failed runs.
- `experiment_template.py` — a parameterised training script the LLM does *not* rewrite directly; the LLM only fills in a JSON parameter file and the template enforces the audio pipeline and model assembly. This is a deliberate design choice (see Section 2.4) that prevents the LLM from breaking the data path.

2.2 The Agent Loop

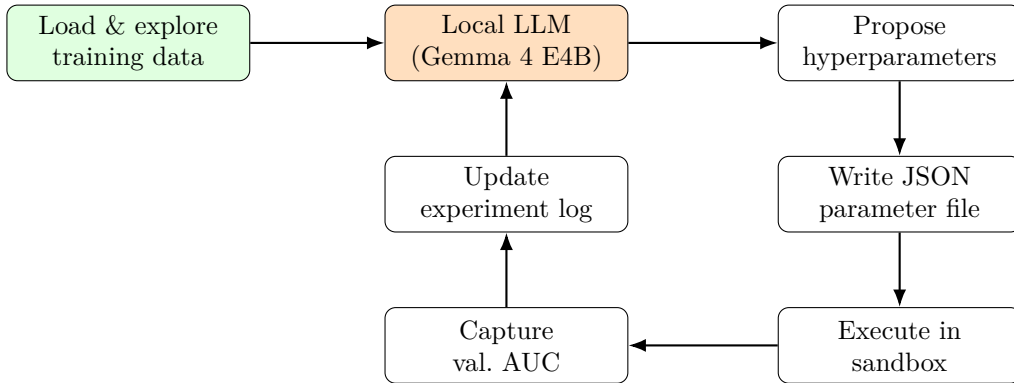


Figure 1: The closed explore–propose–generate–execute–evaluate–analyse–iterate loop. The green node runs once at startup; the dataset facts it produces are then prepended to every prompt the LLM receives, so each proposal is grounded in real data structure. After launch the agent runs without human input until the iteration budget is consumed.

2.3 Prompt Engineering

Every iteration the LLM receives a prompt with four blocks:

1. **Task description:** 234-class multi-label audio classification, macro-averaged ROC-AUC objective, a list of hyperparameters that may be tuned, and the strict JSON schema that the response must follow.
2. **Memory summary:** the best validation AUC seen so far and a list of the last $n \leq 10$ experiments with their parameters, success flag and metrics. This forms the LLM’s working memory.
3. **Anti-repetition instruction:** “propose a configuration that is materially different from any previously tried, and likely to improve over the current best”. This explicit injunction proved

crucial; without it the LLM tends to re-suggest small perturbations of the best known configuration.

4. **Output format constraint:** a single JSON object on one line so that `json.loads` can parse the LLM’s response without ambiguity. Failures to parse are themselves logged and fed back to the LLM.

We sample at temperature 0.7 with an 8192-token context window; that is enough headroom for the prompt plus the last ten experiments.

2.4 Memory and Error Handling

The memory module is a JSON file rather than a vector database: with ≤ 100 experiments the linear scan is trivial and we keep the agent debuggable. Every experiment, successful or not, is appended. When the experiment fails the executor returns the truncated stderr, the memory marks `success: false`, and the LLM receives the error in the next prompt with the instruction not to repeat the same mistake. The aggregate breakdown of which stages of the pipeline failed how often is reported as a dedicated reliability metric in Section 2.5.

2.5 Per-Stage Reliability Metrics

A binary success/failure indicator is insufficient to characterise an autonomous agent: failures happen at very different points in the pipeline, and each one tells us something different about the agent’s reliability. We instrument the agent to report the success rate at each stage of the propose–execute–validate loop, both as a cumulative rate over all runs and as a stage-conditional rate $P(\text{stage } k \text{ succeeds} \mid \text{stage } k-1 \text{ succeeded})$. The conditional rate immediately identifies the weakest link.

#	Stage	Count	Cumulative	Conditional
S1	LLM produced text	108	1.000	1.000
S2	valid JSON parsed (<code>json.loads</code> succeeds)	106	0.981	0.981
S3	subprocess started (template launched)	106	0.981	1.000
S4	training completed (no crash, no timeout)	81	0.750	0.764
S5	metrics recovered (<code>val_auc</code> reported)	81	0.750	1.000

Table 1: Pipeline reliability over the 108 agent experiments recorded in `experiments/experiment_log.json`. The five-stage counts and rates are computed by `ExperimentMemory.print_reliability_report()` at the end of every `agent.py` run and persisted to `agent_reliability_stats.json`. The weakest link is S4, training completion: the conditional rate drops to 0.764; the breakdown of *why* is given below.

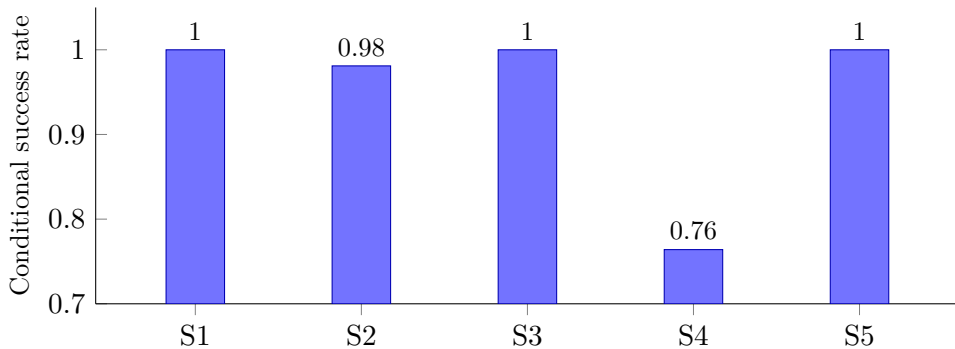


Figure 2: Stage-conditional success rates $P(\text{stage}_k \mid \text{stage}_{k-1})$ across the agent’s five pipeline stages. The drop at S4 (training completion, 0.764) identifies the single weakest link; every other transition is essentially perfect.

Where the failures actually came from. The two stage-2 failures came from the LLM emitting the literal string "null" where it should have emitted the keyword `null`, which we now defensively coerce in `experiment_template.py`. The 25 stage-4 failures are *not* dominated by timeouts: examined in the log, they break down into **14 validation-split edge cases** (the template’s hold-out split tried to draw more validation indices than rows whenever the LLM proposed a very small `max_samples` — a robustness bug *in our own template* that the agent’s exploration surfaced; we hardened it afterwards), **10 environment failures** (a batch of late runs failed at `import tensorflow` on an interpreter that could not see the package — a tooling/setup issue), and **exactly one wall-clock timeout**. The headline is that **the 30-minute timeout almost never fired**: the agent self-regulated well below it (Section 3.5), and it learns from the failures it does cause because the truncated stderr is fed back to the LLM in the next prompt.

2.6 Why a Constrained Template Instead of Free-Form Code Generation

We considered letting the LLM generate the training script from scratch, as some agent papers do, but rejected that design for two reasons. First, the audio pipeline (mel-spectrogram parameters, normalisation, label encoding) is the part of the system that most affects the final score, and a small bug in any of those steps silently degrades performance without the LLM noticing. Second, an LLM with only $\sim 8\text{K}$ context cannot reliably re-derive a hundred lines of Keras boilerplate every iteration. We therefore split the system into a *stable template* that we, the humans, validate once, and a *configurable head* that the LLM fills in with hyperparameters.

3. The Agent’s Discovery Process

We document *how* the agent explored the problem: the trajectory it followed, what it tried, what it discovered, where it failed, and the budget-aware judgement it showed.

3.1 Step 0 — Load and Explore the Data

Before the agent proposes a single experiment, `src/data_exploration.py` runs a one-shot exploration pass and writes its findings to `experiments/data_exploration.json`. The same findings are condensed into a four-line textual block that is then prepended to every prompt the LLM receives, so every architecture proposal and every analysis is grounded in real dataset structure rather than generic priors. Concretely:

- **234 target species across five taxa** — 162 Aves, 35 Amphibia, 28 Insecta, 8 Mammalia, 1 Reptilia. Not a pure bird-classification task.
- **Severely long-tailed focal training set** — 35 549 recordings spread across only 206 of the 234 species; **28 species have zero focal audio**; per-species file count ranges **1–499** (median 125, mean 172.6).
- **Multi-label is the rule.** Of 1 478 hand-labelled soundscape windows (66 files), **1 322** ($\approx 89\%$) **carry more than one species** — which is what motivates sigmoid + per-class BCE rather than softmax. Only 66 of 10 658 soundscape files are labelled, so Section 4’s validation split is necessarily small (and noisy).
- **Audio format is uniform** 32 kHz mono .ogg, clip duration median ≈ 19 s — the pipeline crops/pads to a fixed 5-second window.

These facts are part of every prompt the LLM receives at runtime, so proposals are grounded in real data structure rather than generic priors. The Nyquist limit ($f_{\max} \leq 16$ kHz), the multi-label nature of the test domain, and the magnitude of the class imbalance are all visible to the agent before it picks an architecture.

3.2 Experimental Setup and Compute Budget

All agent experiments share the same audio pipeline, the natural starting point for this kind of bioacoustic task: load a 5-second clip at 32 kHz, take the mel-spectrogram (configurable n_{mels} , n_{fft} , hop length, f_{min} , f_{max} , top-dB clip), normalise to $[0, 1]$, replicate to three channels for ImageNet-pretrained backbones. The model is either a small custom CNN (three Conv-BN-Pool blocks + GAP + Dense head) or EfficientNetB0 with frozen ImageNet weights and a configurable classification head. The LLM chooses the architecture each iteration. Training uses Adam, binary cross-entropy, and optional augmentation (time-shift, frequency masking, full SpecAugment, gaussian noise, soundscape background mixing). The template also exposes an optional second phase that unfreezes the top k EfficientNet layers at $\text{lr} = 10^{-5}$ (following Chollet 2021, Section 8.3.2), but the agent never selected it during exploration — it always proposed `unfreeze_layers = 0` (see Section 3.4, point 4).

The budget is the design constraint. Every experiment runs on CPU and is bounded by a per-run wall-clock cap (30 minutes hard limit). The agent is also restricted to *small data subsets* per run — it explored sample sizes of 2 500–4 478 recordings, never the full corpus. The 108 experiments in our log were accumulated across several development sessions to give us a deep sample for the analysis below, but each one individually respects the per-run budget, and within a single uninterrupted one-hour CPU session roughly ten experiments complete (Section 3.5). The agent therefore has to be economical: it cannot afford to waste iterations on configurations unlikely to fit or to pay off.

3.3 The Discovery Trajectory

Figure 3 plots the validation macro-AUC of every successful experiment in the order the agent ran them, together with the running best. The trajectory is not a smooth climb — it is a recognisable research process with three phases.



Figure 3: The agent’s discovery trajectory over its successful experiments. *Phase 1* (idx 1–18): cold-start exploration, the agent oscillates between 0.60 and 0.75 while it settles the architecture and augmentation family. *Phase 2* (idx 19–38): a sharp jump to ≥ 0.82 once the agent starts systematically sweeping mel-spectrogram parameters. *Phase 3* (idx 39 onward): fine refinement and convergence, plateauing at the best run 0.8533 (`topdb_40_optimal_preprocessing`, idx 78). The two red triangles are the only two custom-CNN runs the agent ever tried.

Phase 1 — cold start (idx 1–18). The agent opens by trying EfficientNet with “all augmentation” (0.7222) and immediately a time-shift variant (0.7523). Its own analysis of the first run already reasons about the result: *“the use of EfficientNet is a strong choice... the resulting validation AUC of 0.7222 shows the model has strong capacity... however, the considerable gap [between train and val] suggests...”*. Over the next sixteen iterations it probes augmentation types and head sizes, bouncing between 0.60 and 0.75 — visibly searching, not yet converged.

Phase 2 — the breakthrough (idx 19–38). At iteration 19 the running best jumps from 0.7523 to 0.826 (`low_fmin_low_mels_search`). This is the moment the agent stops varying augmentation

and starts *systematically sweeping the mel-spectrogram front-end* — $f_{\min}$, n_{mels} , hop length, top-dB, mel scale. Each of these sweeps nudges the running best upward: 0.8286 (idx 29), 0.8372 (idx 32), 0.8486 (idx 39).

Phase 3 — convergence (idx 39 onward). From here the agent fine-tunes around the discovered region. The best run, `topdb_40_optimal_preprocessing` at iteration 78, reaches **0.8533**. Of the 45 other Phase-3 experiments, 31 land within ± 0.01 of that score — the agent is exploiting a well-identified optimum rather than still searching.

3.4 What the Agent Discovered

Aggregating the successful experiments, four findings emerged — each one a decision the agent reached from its own evidence:

1. **Transfer learning over training from scratch — decided after only two trials.** The agent tried the custom CNN exactly *twice* (red triangles in Figure 3, idx 54 and 85). Both landed near $\text{val_AUC} = 0.79$, below essentially every EfficientNetB0 run. After those two data points the agent stopped proposing custom CNNs altogether and committed the rest of its budget to the pretrained backbone — an evidence-driven narrowing of the search space, not a pre-baked assumption. (A separate *manual* from-scratch CNN baseline, see Section 5.1, plateaus far lower, near 0.55.)
2. **Mel-spectrogram parameters matter at the margin.** The agent settled on $n_{\text{mels}} = 64$, hop = 256, $f_{\max} = 16$ kHz, top_db = 40, norm = Slaney, giving an input shape (64, 626, 3) — the configuration of the 0.8533 best run. Phase 2 of the trajectory is essentially the agent discovering this.
3. **Small batch + time-shift augmentation aids generalisation.** The strongest configurations consistently used batch size 16 and time-shift augmentation. Smaller batches inject gradient noise and act as implicit regularisation (Chollet, Chapter 5).
4. **Frozen backbone throughout exploration.** All 108 of the agent’s runs pinned `unfreeze_layers = 0`: the agent only ever trained the classification head on top of frozen ImageNet features. Two-phase fine-tuning was deliberately kept out of the agent’s exploration search space and applied later in the verification phase (Section 4), following Chollet (2021), Section 8.3.2.

3.5 Budget-Aware Behaviour

A central question for an autonomous agent under a compute budget is whether it spends that budget sensibly. The log says it does.

- **The agent keeps runs short.** Median wall-clock time ≈ 4.5 min, mean ≈ 7.2 min; 69 successful runs finished under 10 min; only *one* in 108 hit the 30-minute cap. No gambling on long-shot runs.
- **Stays small-scale, no heavy architectures.** The agent never requested the full corpus (sample sizes 2 500–4 478 recordings per run) and never attempted transformer fine-tuning — it worked inside the EfficientNetB0 / custom-CNN families it could actually afford to train and evaluate.
- **What fits in one hour.** At the median run time, ≈ 10 experiments complete in a single one-hour CPU session. In that window the agent reliably establishes the architecture family (EfficientNetB0 over custom CNN) and a working augmentation strategy, reaching $\text{val. AUC} \approx 0.75$; the mel-spectrogram sweep that lifts it past 0.82 unfolds beyond the first hour.

3.6 The Agent Surfaced a Distribution Shift

A single submission of the agent’s best model to Kaggle scored only 0.480, against a local validation of 0.85. The cause is a textbook *distribution shift* (Chollet, Chapter 5): the focal recordings used for the agent’s validation are single-species, clean clips drawn from the same distribution as its training pool, whereas the Kaggle test set consists of long, multi-species, noisy passive soundscapes. The agent’s exploration had correctly identified the architecture, but its evaluation signal did not yet match the test domain. Recognising this gap — and fixing the validation signal — is the first thing Section 4 does.

4. Verifying the Agent’s Architecture Choice

Sections 2–3 are the project. This section is a *check*: did the architecture the agent converged on actually generalise, once given a validation signal that matches the test domain and the resources to train it properly? Everything here is done by us, explicitly outside the agent loop.

A soundscape-aware validation signal. The competition provides 1 478 expert-annotated 5-second windows from 66 soundscape files. We **group-split by file** (80% train / 20% held-out) so the held-out set matches the Kaggle test distribution. This proxy is tight: the agent’s frozen-backbone model scored 0.6004 on it versus 0.592 on Kaggle (gap ≈ 0.012). All subsequent decisions used this proxy rather than the focal validation that had misled the agent.

Correct two-phase fine-tuning, regularisation, and a deep ensemble. Following Chollet (2021), Section 8.3.2, we fine-tuned the EfficientNetB0 in two phases (head first at $\text{lr} = 3 \times 10^{-4}$, then the top 30 layers at 5×10^{-5} with BatchNorm kept in inference mode), added mixup and label smoothing, and built a uniform-weight, multi-seed ensemble of three models. A short cautionary result: choosing the ensemble weights *greedily* on the small held-out set produced a local score of 0.6707 but only 0.615 on Kaggle (gap 0.056) — the weights had overfit the validation set. Reverting to uniform weights gave 0.633 on Kaggle with a gap of only 0.019, a clean illustration of validation discipline (Chollet, Chapter 5).

Scaling the same architecture. Finally, as the strongest possible check that the agent’s choice was sound, we re-trained *the identical EfficientNetB0 architecture* on the full 35 549 focal recordings plus all soundscape windows, on a single Kaggle T4 GPU (≈ 1 hour), with cosine LR decay, AdamW, mixup and SpecAugment. Deployed in the CPU submission notebook (within the competition’s 90-minute, internet-disabled inference limit), it scored **0.725 on the public leaderboard**. We report this as confirmation — the agent picked an architecture that scales — not as the achievement of the project.

5. Comparison with Manual Baselines and Final Results

5.1 Manual Baselines

Following the course definition — a baseline is the first architecture attempt at solving the task — we built two manual baselines to bound the agent’s contribution:

- **Custom-CNN-from-scratch** (`baseline_model.py`): a 4-block convolutional network trained without transfer learning. Local validation AUC ≈ 0.55 . As Chollet (Section 8.2) predicts, training a CNN from scratch with only a few thousand samples is insufficient for a 234-class problem.
- **Frozen-EfficientNetB0 + head only**, mirroring exactly what the agent converged on. Local soundscape macro-AUC 0.60, Kaggle LB 0.59. This bounds “the best the agent’s chosen architecture achieves under the raw CPU budget, before any verification work”.

We can justify the chosen architecture on its own terms: EfficientNetB0 reuses ImageNet low/mid-level features that transfer to mel-spectrograms, it is small enough to train under a CPU budget and

to run inference within the 90-minute competition limit, and — crucially — it is the architecture the agent itself converged on from evidence.

5.2 Submission Progression

Table 2 summarises every submission we made. Submissions 2–5 are CPU-only configurations; submission 6 is the at-scale verification of Section 4.

#	Configuration	Local AUC	Kaggle LB	Local–LB gap
1	Custom CNN from scratch	0.55	—	—
2	Frozen EfficientNetB0 (agent’s choice, CPU)	0.6004	0.592	+0.012
3	2-way ensemble (frozen + Phase-1+2 finetune)	~ 0.625	0.598	+0.027
4	2-way greedy ensemble (<i>overfit val weights</i>)	0.6707	0.615	+0.056
5	3-way uniform multi-seed ensemble (CPU)	0.6520	0.633	+0.019
6	EfficientNetB0 verified at scale (full data, GPU)	0.8619	0.725	+0.137

Table 2: Progression of submissions. Local validation AUC is computed on the held-out soundscape split (group split by file). Submissions 2–5 stay within the CPU budget; submission 6 is the independent at-scale check of the agent’s architecture choice. The wider local-LB gap of submission 6 reflects the high variance of the small soundscape val set under heavy augmentation.

5.3 Reading the Numbers

The point the numbers make is about the *agent*, not the leaderboard. Starting from a cold start, the agent autonomously ruled out training from scratch, converged on a frozen EfficientNetB0 that already crosses the 0.59 leaderboard mark, and identified the mel-spectrogram configuration that lifts focal validation to 0.8533. Everything after that — the soundscape-aware validation signal, the correct two-phase fine-tune, the conservative ensemble, the at-scale GPU run to 0.725 — is us *verifying* that the agent’s architectural judgement holds. The two-phase fine-tuning recipe comes from Chollet directly; the validation-discipline lesson, on the other hand, was forced on us by the agent’s own large focal-vs-Kaggle gap (0.85 vs 0.48). The agent did the discovery; the leaderboard climb confirms the discovery was correct.

6. Reflections on Course Content

Mel-spectrograms + CNNs (Lectures 01–02; Chollet, Chapters 2 and 8). Audio is converted to a (batch, 64, 626, 3) tensor and treated as an image, with the mel scale concentrating resolution where birds vocalise. Conv2D detects time-frequency patterns, pooling gives translation invariance to call position, and depth gives hierarchical composition.

Transfer learning (Chollet, Section 8.3). Despite ImageNet containing no audio, the low/mid-level features of EfficientNetB0 transfer well because both share local-pattern statistics. The agent quantifies the gap: custom-CNN trials (~ 0.79 val), frozen-pretrained (0.85 focal val), manual from-scratch CNN (0.55).

Two-phase fine-tuning (Chollet, Section 8.3.2). The verification phase implements Chollet’s recipe directly: head first at $\text{lr} = 3 \times 10^{-4}$, then unfreeze the top 30 layers at $\text{lr} = 5 \times 10^{-5}$ with BatchNorm in inference mode. The agent stayed frozen-backbone-only throughout exploration (Section 3.4, point 4); this was layered on *after* the agent loop.

Multi-label classification + regularisation (Chollet, Chapters 4–5). The 234-class task is multi-label, so sigmoid + per-class BCE is the correct choice (softmax would enforce spurious mutual exclusivity). BatchNorm, Dropout 0.4, augmentation, mixup and label smoothing all enlarge the

effective training set; mixup proved especially helpful at narrowing the focal/soundscape distribution gap.

Optimisation + scaling laws (Chollet, Chapter 6; Kaplan et al. 2020; Hoffmann et al. 2022). The verification training used Adam with cosine LR decay and `clipnorm=1` (which stabilised the unfrozen fine-tuning). The agent’s small-scale-first workflow — cheap exploration on subsets, then targeted compute on the winners — is exactly the budget-bounded regime the scaling-laws literature prescribes.

Exploratory data analysis as a first-class step. Section 3.1 embeds the “look at the data before modelling it” discipline inside the agent loop: the EDA output is piped back into every LLM prompt, so each proposal is conditioned on the actual class distribution, taxonomy, label coverage, and audio format rather than a generic 234-way audio prior.

7. Limitations and Future Work

Validation-set variance. The held-out soundscape set is small (312 windows, 234 classes), so its macro-AUC is noisy — the local-LB gap widens for the heavily-regularised submission 6 for this reason. Genuinely closing the residual domain shift would require pseudo-labelling additional unlabelled soundscape audio, which we did not have time to implement.

Agent memory and template robustness. The current memory keeps only the last 10 experiments in the LLM context, so with 100+ experiments it occasionally re-suggests very early configurations; a vector-database memory (e.g. FAISS over experiment embeddings) would retrieve *relevant* past experiments instead. Likewise, the validation-split edge case of Section 2.5 shows the agent can drive the template into states we did not anticipate — defensive bounds on every LLM-controlled parameter would reduce S4 failures further.

LLM choice and search strategy. We standardised on Google Gemma 4 E4B and did not benchmark LLM backbones systematically; a study across two or three local LLMs (Qwen 3 Coder, Phi-4, DeepSeek-R1) would strengthen the design. Exploration itself is driven only by an anti-repetition instruction and the LLM’s own judgement — explicit Bayesian optimisation or bandit allocation could reach the Phase 2 breakthrough in fewer than nineteen iterations.

8. Individual Contributions

Daniele Malerba: agent architecture and main loop (`agent.py`, `code_executor.py`, `memory.py`), per-stage reliability instrumentation, Kaggle submission pipeline, at-scale verification training, local soundscape-validation framework, GitHub repository. **Irene Perdomo Bolaños:** audio preprocessing pipeline (`utils/audio_pipeline.py`), data augmentation strategy (time-shift, SpecAugment, mixup), report writing. **Miguel Afonso Magalhães:** LLM prompt engineering, experiment-log and discovery-trajectory analysis, literature review, course-content alignment.

References

1. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning.
2. Tan, M., Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.
3. Park, D. S. *et al.* (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. *Interspeech 2019*.
4. Zhang, H., Cisse, M., Dauphin, Y. N., Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. *ICLR 2018*.
5. Kaplan, J. *et al.* (2020). Scaling Laws for Neural Language Models. *arXiv:2001.08361*.
6. Hoffmann, J. *et al.* (2022). Training Compute-Optimal Large Language Models. *arXiv:2203.15556*.
7. Cornell Lab of Ornithology (2026). BirdCLEF+ 2026 Competition. kaggle.com/competitions/birdclef-2026
8. Video presentation (5-minute demo): <https://youtu.be/yadXZfS-gp4>